

Expressões Regulares

Linguagens Formais e Autômatos

Engenharia de Software
Profª Kadidja Valéria

O problema dos padrões

Como descrever, de forma compacta, todas as cadeias que obedecem a uma regra?

LFA-2026-001

Enumerar todos os códigos seria inviável

Uma expressão descreve um conjunto de palavras

A linguagem vem antes da sintaxe

Objetivos de aprendizagem

Interpretar expressões regulares e explicar a linguagem representada

Construir expressões a partir de uma especificação

Testar casos aceitos, rejeitados e de fronteira

Relacionar Regex, linguagens regulares, DFA e NFA

Conceitos fundamentais

Objetos

Alfabeto: conjunto finito de símbolos

Palavra: sequência finita

ϵ : palavra de comprimento zero

Operações

União: escolha entre linguagens

Concatenação: sequência

Fechamento de Kleene: zero ou mais

Linguagem e fechamento de Kleene

Para $\Sigma = \{0,1\}$, uma linguagem é qualquer subconjunto de Σ^* .

$$L^* = \{ \epsilon, a, aa, aaa, \dots \}$$

A estrela sempre permite zero repetições

ϵ é uma palavra. $\emptyset$ é um conjunto sem palavras

Definição formal

$\emptyset$, ε e cada símbolo de Σ formam expressões regulares

Se r e s são expressões, então $(r|s)$, (rs) e r^* também são

$$L(r|s) = L(r) \cup L(s)$$

$$L(rs) = L(r)L(s) \text{ e } L(r^*) = L(r)^*$$

Primeiras expressões

Expressão

a representa $\{a\}$

a^* inclui ϵ , a , aa , ...

a^+ exige uma ou mais ocorrências

$a|b$ representa $\{a,b\}$

Observe o agrupamento

ab^* repete apenas b

$(ab)^*$ repete o bloco ab

$(ab)^*$ aceita ϵ e $abab$

$(ab)^*$ rejeita a e abb

Operadores de repetição

Operadores

r^* significa zero ou mais

r^+ equivale a rr^*

$r?$ equivale a $r|\epsilon$

Quantificadores

$\{n\}$: exatamente n

$\{n,m\}$: de n a m

$\{n,\}$: no mínimo n

Classes, intervalos e âncoras

Classes

[abc] escolhe um caractere

[a-z] representa uma minúscula

[0-9] representa um algarismo

Posição no texto

^ exige o início da entrada

\$ exige o fim da entrada

^...\$ valida a palavra inteira

Ponto e escape

Em muitos motores, o ponto reconhece quase qualquer caractere. O escape indica um ponto literal.

a.b versus a\b

a.b pode reconhecer a7b

a\b reconhece literalmente a.b

Teoria clássica e motores Regex

Teoria formal

União, concatenação e estrela formam o núcleo

O poder corresponde aos autômatos finitos

Classes e quantificadores são abreviações

Programação

Motores acrescentam âncoras e grupos

A sintaxe varia entre ferramentas

Retroreferências podem superar o poder regular

Exemplo 1: palavras binárias

Especificação: palavra não vazia formada somente por 0 e 1.

$^+[01]+\$$

Aceita: 0, 1, 101, 00110

Rejeita: ϵ , 102, a01

ER clássica: $(0|1)(0|1)^*$

Exemplo 2: termina em 1

O prefixo pode ser qualquer sequência binária. O último símbolo deve ser 1.

$$^{[01]^*1\$}$$

Aceita: 1, 01, 101, 0001

Rejeita: ϵ , 0, 110

ER clássica: $(0|1)^*1$

Exemplo 3: números inteiros

Sinal opcional seguido por uma sequência não vazia de algarismos.

$^{+|-}?[0-9]^+$

Aceita: 0, 42, -8, +2026

Rejeita: +, --2, 3.14, 12a

Exemplo 4: código de disciplina

Três letras maiúsculas, hífen e quatro algarismos.

$^[A-Z]{3}-[0-9]{4}\$$

Aceita: LFA-2026, ESW-0102

Rejeita: LF-2026, lfa-2026, LFA2026

Exemplo 5: identificadores

O primeiro caractere possui uma regra diferente dos demais.

`^[A-Za-z_][A-Za-z0-9_]*$`

Aceita: x, _total, nota2

Rejeita: 2nota, valor-final, nome completo

Regex, linguagens regulares e autômatos

O Teorema de Kleene estabelece a equivalência de poder

Uma ER pode gerar um ϵ -NFA pela construção de Thompson

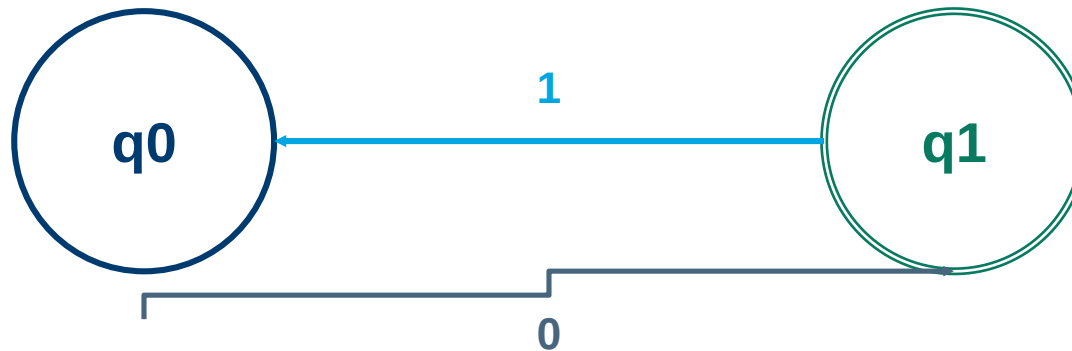
A construção dos subconjuntos converte o NFA em DFA

A eliminação de estados converte um autômato em ER

DFA para palavras que terminam em 1

0 ou último símbolo 0

último símbolo 1



Em qualquer estado: 0 leva a q_0 e 1 leva a q_1

Erros frequentes

Problema

Confundir $*$ com $+$
Esquecer ϵ
Omitir agrupamentos
Confundir $.$ com $\backslash$.
Validar sem âncoras

Prevenção

Analise se vazio pertence à linguagem
Identifique o bloco repetido
Teste entradas quase válidas
Confirme o motor utilizado
Justifique cada subexpressão

Exercício guiado

1. Sobre $\{0,1\}$, descreva palavras que terminam em 00
 2. Sobre $\{a,b\}$, descreva palavras com exatamente dois a
 3. Crie um identificador com duas maiúsculas, três algarismos e uma minúscula opcional
- Registre a linguagem, exemplos aceitos, rejeitados e sua Regex

Resolva antes de testar na ferramenta. Este material não contém gabarito.

Desafio: matrícula acadêmica

Formato: CURSO-ANO-NÚMERO-TURNO

CURSO: CCO, ESW ou SIS

ANO: 2024 a 2029

NÚMERO: exatamente quatro algarismos

TURNO: M, T ou N

Use hífens entre os blocos e não permita caracteres extras

Produza a Regex, justifique cada bloco e crie novos casos de teste.

Atividade no Regex Learn

Construa a expressão a partir das regras

Teste todos os exemplos válidos e inválidos

Crie casos de fronteira e entradas quase corretas

Quando um teste falhar, identifique a regra mal representada

Entregue a Regex, a tabela de testes e a justificativa

Questões para reflexão

Toda expressão regular formal representa uma linguagem regular?

Toda linguagem regular possui uma expressão regular?

Como uma Regex se relaciona com DFA e NFA?

Quais extensões de motores não pertencem à definição clássica?

Por que $\{a^n b^n \mid n \geq 0\}$ não pode ser reconhecida por um DFA?

Síntese

Notação

- | escolha
- * zero ou mais
- + uma ou mais
- ? zero ou uma
- () agrupamento

Método

- Defina a linguagem
- Separe a palavra em blocos
- Traduza ordem, escolha e repetição
- Teste casos positivos e negativos
- Relacione a expressão ao autômato

Conte conosco!

Muito obrigada!



www.udf.edu.br